Reviewer Pack — Minimal Rank of Quasi-group Representations vs DPLL Search T…

Entry b0e9cb0b8133 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 18:23:05 UTC

Minimal Rank of Quasi-group Representations vs DPLL Search Tree Width for k-CNF

Entry ID: `b0e9cb0b8133` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 18:23:05 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

> ['For every quasi-group G, the minimal rank of a faithful representation of G over the two-element Boolean algebra equals the maximum height of any DPLL search tree for k-CNF formulas with at most n clauses.', 'Equivalently, for all instances of k-CNF with n clauses, if G is a quasi-group representing the clause set, then the minimal rank of G is equal to the maximum depth of the corresponding DPLL search tree.', 'For all n ≤ 40 and all k ≥ 3, there exists an instance of k-CNF such that the above equality does not hold.']

Rationale (proposer's reasoning):

> ['Quasi-groups are algebraic structures that generalize groups and may provide a different perspective on computational complexity. The minimal rank of a quasi-group representation could potentially capture aspects of the combinatorial complexity of SAT instances that are not captured by standard group representations.', 'The connection between quasi-group theory and DPLL search trees suggests that properties of quasi-groups might expose new insights into the structure of SAT solutions, potentially leading to improved algorithms for solving SAT problems.']

Taxonomy category: `Quasi_group_theory_vs_DPLL_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `87fc7f4d5df9170a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the minimal rank of quasi-group representations and the maximum depth of DPLL search trees from 30 independent seeds is ≥ 0.8, and the mean absolute difference between these two metrics does not exceed 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `minimal rank quasi-group representation AND DPLL search tree width` - `faithful representation Quasi-group Boolean algebra equalling maximum height DPLL search tree` - `k-CNF clause set quasi-group representation minimal rank comparison DPLL search tree depth`

Top relevant hits considered: - [http://arxiv.org/abs/2401.11576v6] Quantum Architecture Search with Unsupervised Representation Learning - [http://arxiv.org/abs/1006.5752v1] Solomon's induction in quasi-elementary groups - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^aq - [http://arxiv.org/abs/1005.0140v4] Representations of hom-Lie algebras - [http://arxiv.org/abs/1807.11134v3] Faithful irreducible representations of modular Lie algebras - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/2202.03982v3] Depth zero representations over $\overline{\mathbb{Z}}[\frac{1}{p}]$ - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for j in range(n):
        i_max = -1
        for i in range(rank, m):
            if A[i][j] != 0:
                i_max = i
                break
        if i_max == -1:
            continue
        A[rank], A[i_max] = A[i_max], A[rank]
        pivot = A[rank][j]
        for i in range(rank + 1, m):
            factor = Fraction(A[i][j], pivot)
            for j2 in range(n):
```

```
                A[i][j2] -= factor * A[rank][j2]
        rank += 1
    return rank

def matrix_rank(matrix):
    if not matrix or not matrix[0]:
        return 0
    m, n = len(matrix), len(matrix[0])
    A_copy = [row[:] for row in matrix]
    return gaussian_elimination(A_copy)

def construct_quasi_group(n):
    G = {}
    elements = list(range(1, n + 2))
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            G[(i, j)] = (i * j) % (n + 1)
    return G

def dpll_search_tree(k, clauses):
    if not clauses:
        return 0
    literals = set()
    for clause in clauses:
        literals.update(clause)
    for literal in literals:
        new_clauses = [clause for clause in clauses if literal not in clause and -literal not in c
        depth = dpll_search_tree(k, new_clauses) + 1
        if depth > k:
            return depth
    return k

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5):  # Sample 5 instances per size
            k = random.randint(3, 6)
            clauses = [[random.randint(1, n) for _ in range(random.randint(1, k))] for _ in range(
            G = construct_quasi_group(n)
            rank = matrix_rank(G)
            depth = dpll_search_tree(k, clauses)
            results.append((rank, depth))

    if not results:
        return {
            "metric_name": "Rank vs DPLL Depth",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    ranks = [r for r, _ in results]
```

```
    depths = [d for _, d in results]
    correlation_coefficient = sum((r - mean(ranks)) * (d - mean(depths)) for r, d in results) / ma
    mean_rank = mean(ranks)
    mean_depth = mean(depths)
    max_abs_diff = max(abs(r - d) for r, d in results)

    return {
        "metric_name": "Rank vs DPLL Depth",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": abs(correlation_coefficient) >= 0.8 and max_abs_diff <= 3,
        "counterexample": ""
    }

def mean(values):
    return sum(values) / len(values)

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = mean([r["metric_value"] for r in results])
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(resu
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE No seeds tested")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0b879b3.py", line 113, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0b879b3.py", line 76, in run_trial
    rank = matrix_rank(G)
           ^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0b879b3.py", line 39, in matrix_rank
    if not matrix or not matrix[0]:
                         ~~~~~~^^^
```

```
KeyError: 0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and mean absolute difference as required by the support condition. | next: Review the code for potential bugs that could cause a crash during execution and ensure it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11819
2	propose	ollama_remote	glm4:latest	0	0	10755
3	propose	ollama_remote	glm4:latest	0	0	10899
4	preregistration	ollama_remote	glm4:latest	0	0	5641
5	novelty	ollama_remote	glm4:latest	0	0	4718
6	novelty	ollama_remote	glm4:latest	0	0	5796
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21667
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14132
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11181
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12909
11	judge	ollama_remote	glm4:latest	0	0	12859

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122375 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b0e9cb0b8133.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0e9cb0b8133.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b0e9cb0b8133.tar.gz` (if generated)